

LAPORAN PRATIKUM

STRUKTUR DATA

“Pratikum Pekan 5”

Disusun Oleh:

Rafikhul Ramadhan

2511533012

Dosen Pengampu : Dr. Wahyudi, S.T, M.T.

Asisten Praktikum : Muhammad Galis Avero



DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS

2025

KATA PENGANTAR

Segala puji dan syukur saya panjatkan ke hadirat Allah SWT atas limpahan rahmat, taufik, dan karunia-Nya sehingga penulis dapat menyelesaikan laporan praktikum mata kuliah Struktur Data ini dengan baik dan tepat waktu.

Laporan praktikum ini disusun sebagai salah satu bentuk pemenuhan tugas akademik, sekaligus sebagai sarana untuk memperdalam pemahaman penulis terhadap konsep-konsep dasar dalam struktur data. Melalui praktikum ini, saya mempelajari berbagai materi penting seperti penggunaan array, stack, queue, linked list, serta bagaimana mengimplementasikannya dalam bahasa pemrograman. Selain itu, kegiatan praktikum juga melatih kemampuan berpikir logis, sistematis, dan terstruktur dalam menyelesaikan permasalahan.

Saya menyadari bahwa dalam penyusunan laporan ini masih terdapat berbagai kekurangan, baik dari segi penulisan maupun isi. Oleh karena itu, saya sangat mengharapkan kritik dan saran yang membangun guna perbaikan di masa yang akan datang.

Pada kesempatan ini, saya juga ingin menyampaikan rasa terima kasih kepada dosen pengampu, asisten praktikum, serta teman-teman yang telah memberikan bimbingan, dukungan, dan bantuan selama proses praktikum hingga penyusunan laporan ini.

Akhir kata, semoga laporan praktikum ini dapat memberikan manfaat bagi penulis khususnya, serta bagi para pembaca pada umumnya. Penulis berharap laporan ini juga dapat digunakan sebagai referensi dalam memahami materi praktikum, baik pada mata kuliah Struktur Data maupun praktikum lainnya.

Padang, 5 April 2026

Rafikhul Ramadhan

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI	ii
BAB I PENDAHULUAN	1
1.1 Latar Belakang.....	1
1.2 Tujuan	1
1.3 Manfaat	1
BAB II PEMBAHASAN.....	2
2.1 Langkah Kerja.....	2
2.2 QueueArray_2511533012	2
2.3 QueueArrayDriver_2511533012	3
2.4 QueueLinkedList_2511533012.....	6
2.5 ReverseData_2511533012.....	7
2.6 IterasiQueue_2511533012	Error! Bookmark not defined.
BAB III KESIMPULAN	11
3.1 Ringkasan.....	11
3.2 Kesimpulan.....	11
DAFTAR PUSTAKA	12

BAB I

PENDAHULUAN

1.1 Latar Belakang

Pada praktikum pekan ke-5, mahasiswa mempelajari konsep **Single Linked List (SLL)** dalam bahasa pemrograman Java. Single Linked List merupakan struktur data linear yang terdiri dari kumpulan simpul (node), di mana setiap node menyimpan data dan alamat node berikutnya. Struktur data ini digunakan untuk mengatasi keterbatasan array dalam pengelolaan data yang dinamis.

Melalui praktikum ini, mahasiswa mempelajari cara membuat node, menambah data, menghapus data, melakukan traversal, dan mencari data pada linked list. Pemahaman mengenai linked list sangat penting karena menjadi dasar dalam mempelajari struktur data yang lebih kompleks.

1.2 Tujuan

Tujuan dari praktikum pekan ke-5 adalah untuk memahami konsep dasar Single Linked List, mengetahui cara kerja node dan pointer, serta mampu mengimplementasikan operasi-operasi dasar linked list dalam bahasa Java..

1.3 Manfaat

Manfaat dari praktikum ini adalah mahasiswa dapat memahami pengelolaan data dinamis menggunakan linked list, meningkatkan kemampuan logika pemrograman, serta mampu menerapkan operasi penambahan, penghapusan, dan pencarian data pada struktur data linked list.

BAB II

PEMBAHASAN

2.1 Langkah Kerja

Tahapan yang dilakukan dalam pratikum ini adalah sebagai berikut:

1. Menjalankan aplikasi IDE (seperti Eclipse atau IntelliJ IDEA) atau menggunakan text editor yang mendukung bahasa Java.
2. Membuat sebuah project baru dengan nama **pekan5_2511533012**.
3. Menambahkan tiga buah file Java, yaitu NodeSLL_2511533012.java, TambahSLL_2511533012.java, PencarianSLL_2511533012.java, HapusSLL_2511533012.java.
4. Menuliskan kode program sesuai dengan instruksi yang diberikan pada masing-masing file.
5. Melakukan proses kompilasi menggunakan *Java compiler* agar program dapat dijalankan.
6. Menjalankan program untuk mengamati hasil keluaran sesuai dengan logika yang dibuat.
7. Mengevaluasi hasil eksekusi pada *console* serta menarik kesimpulan dari setiap percobaan yang dilakukan.

2.2 NodeSLL_2511533012

```
package Pekan5_2511533012;

public class NodeSLL_2511533012 {

    int data_3012;

    NodeSLL_2511533012 next_3012;

    public NodeSLL_2511533012(int data_3012) {
        this.data_3012 = data_3012;
        this.next_3012 = null;
    }

    public static void main(String[] args) {
        // TODO Auto-generated method stub
    }
}
```

```
}  
}
```

Analisis:

Program ini merupakan class dasar untuk membentuk **node pada Single Linked List**.

Class memiliki atribut data untuk menyimpan nilai dan next untuk menyimpan alamat node berikutnya. Constructor digunakan untuk menginisialisasi data dan mengatur nilai next menjadi null.

Kesimpulan: Program ini berfungsi sebagai dasar pembentukan linked list.

2.3 TambahSLL_2511533012

```
package Pekan5_2511533012;  
  
public class TambahSLL_2511533012 {  
    public static NodeSLL_2511533012  
insertAtFront_3012(NodeSLL_2511533012 head_3012, int value_3012) {  
        NodeSLL_2511533012 new_node = new  
NodeSLL_2511533012(value_3012);  
        new_node.next_3012 = head_3012;  
        return new_node;  
    }  
  
    public static NodeSLL_2511533012  
insertAtEnd_3012(NodeSLL_2511533012 head_3012, int value_3012) {  
        NodeSLL_2511533012 newNode = new  
NodeSLL_2511533012(value_3012);  
        if (head_3012 == null) {  
            return newNode;  
        }  
  
        NodeSLL_2511533012 last = head_3012;  
        while (last.next_3012 != null) {  
            last = last.next_3012;  
        }  
  
        last.next_3012 = newNode;  
        return head_3012;  
    }  
  
    static NodeSLL_2511533012 GetNode_3012(int data_3012) {  
        return new NodeSLL_2511533012(data_3012);  
    }  
}
```

```

        static NodeSLL_2511533012 insertPos_3012(NodeSLL_2511533012
headNode_3012, int position_3012, int value_3012) {
    NodeSLL_2511533012 head_3012 = headNode_3012;
    if (position_3012 < 1)
        System.out.println("Invalid position");
    if (position_3012 == 1) {
        NodeSLL_2511533012 new_node = new
NodeSLL_2511533012(value_3012);
        new_node.next_3012 = head_3012;
        return new_node;
    } else {
        while (position_3012-- != 0) {
            if (position_3012 == 1) {
                NodeSLL_2511533012 newNode =
GetNode_3012(value_3012);
                newNode.next_3012 =
headNode_3012.next_3012;
                headNode_3012.next_3012 = newNode;
                break;
            }
            headNode_3012 = headNode_3012.next_3012;
        }
        if (position_3012 != 1)
            System.out.println("Posisi di luar jangkauan");
        return head_3012;
    }
}

    public static void printList_3012(NodeSLL_2511533012
head_3012) {
        NodeSLL_2511533012 curr = head_3012;
        while (curr.next_3012 != null) {
            System.out.print(curr.data_3012 + "--->");
            curr = curr.next_3012;
        }
        if (curr.next_3012 == null) {
            System.out.print(curr.data_3012);
            System.out.println();
        }
    }

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        NodeSLL_2511533012 head = new NodeSLL_2511533012(2);
        head.next_3012 = new NodeSLL_2511533012(3);
        head.next_3012.next_3012 = new NodeSLL_2511533012(5);
        head.next_3012.next_3012.next_3012 = new
NodeSLL_2511533012(6);

        System.out.print("Senarai berantai awal : ");
        printList_3012(head);
    }
}

```

```

        System.out.print("tambah 1 simpul di depan : ");
        int data_3012 = 1;
        head = insertAtFront_3012(head, data_3012);

        printList_3012(head);

        System.out.print("tambah 1 simpul di belakang : ");
        int data2_3012 = 7;
        head = insertAtEnd_3012(head, data2_3012);

        printList_3012(head);

        System.out.print("tambah 1 simpul ke data 4 : ");
        int data3_3012 = 4;
        int pos_3012 = 4;
        head = insertPos_3012(head, pos_3012, data3_3012);

        printList_3012(head);
    }
}

```

Output (2.2. QueueArray_2511533012.java) :

```

C:\Program Files\Java\jdk-2
> java QueueArray_2511533012
Senarai berantai awal : 2--->3--->5--->6
tambah 1 simpul di depan : 1--->2--->3--->5--->6
tambah 1 simpul di belakang : 1--->2--->3--->5--->6--->7
tambah 1 simpul ke data 4 : 1--->2--->3--->4--->5--->6--->7

```

Analisis :

Program ini digunakan untuk melakukan operasi **penambahan node** pada Single Linked List.

Terdapat beberapa method seperti:

- insertAtFront untuk menambah node di depan
- insertAtEnd untuk menambah node di belakang
- insertPos untuk menambah node pada posisi tertentu

Program juga menampilkan isi linked list menggunakan method printList.

Kesimpulan: Program ini menunjukkan berbagai cara penambahan data pada linked list.

2.4 PencarianSLL_2511533012

```
package Pekan5_2511533012;

public class PencarianSLL_2511533012 {
    static boolean searchKey_3012(NodeSLL_2511533012 head_3012,
int key_3012) {
        NodeSLL_2511533012 curr = head_3012;
        while (curr != null) {
            if (curr.data_3012 == key_3012)
                return true;
            curr = curr.next_3012;
        }
        return false;
    }

    public static void travelsal_3012 (NodeSLL_2511533012
head_3012) {
        NodeSLL_2511533012 curr = head_3012;
        while (curr != null) {
            System.out.print(" " + curr.data_3012);
            curr = curr.next_3012;
        }
        System.out.println();
    }

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        NodeSLL_2511533012 head = new NodeSLL_2511533012(14);
        head.next_3012 = new NodeSLL_2511533012(21);
        head.next_3012.next_3012 = new NodeSLL_2511533012(13);
        head.next_3012.next_3012.next_3012 = new
NodeSLL_2511533012(30);
        head.next_3012.next_3012.next_3012.next_3012 = new
NodeSLL_2511533012(10);
        System.out.print("Penelusuran SLL : ");
        travelsal_3012(head);

        int key_3012 = 30;
        System.out.print("cari data " + key_3012 + " = ");
        if (searchKey_3012(head, key_3012))
            System.out.println("Ketemu");
        else
            System.out.println("Tidak ada");
    }
}
```

Output :

```
Penelusuran SLL : 14 21 13 30 10
cari data 30 = Ketemu
```

Analisis :

Program ini digunakan untuk melakukan **pencarian data pada Single Linked List**.

Method `searchKey` digunakan untuk mencari data tertentu dengan cara menelusuri node satu per satu hingga data ditemukan atau mencapai akhir list. Program juga menggunakan method `traversal` untuk menampilkan seluruh isi linked list.

Kesimpulan: Program ini menunjukkan proses traversal dan pencarian data pada linked list.

2.5 HapusSLL_2511533012

```
package Pekan5_2511533012;

public class HapusSLL_2511533012 {
    public static NodeSLL_2511533012
deleteHead_3012(NodeSLL_2511533012 head_3012) {
        if (head_3012 == null) {
            return null;

            head_3012 = head_3012.next_3012;

            return head_3012;
        }

        public static NodeSLL_2511533012
removeLastNode_3012(NodeSLL_2511533012 head_3012) {
            if (head_3012 == null) {
                return null;
            }
            if (head_3012.next_3012 == null) {
                return null;
            }
            NodeSLL_2511533012 secondLast_3012 = head_3012;
            while (secondLast_3012.next_3012.next_3012 != null) {
                secondLast_3012 = secondLast_3012.next_3012;
            }
            secondLast_3012.next_3012 = null;
            return head_3012;
        }
    }
}
```

```

    }

    public static NodeSLL_2511533012
deleteNode_3012(NodeSLL_2511533012 head_3012, int position_3012) {
    NodeSLL_2511533012 temp = head_3012;
    NodeSLL_2511533012 prev = null;

    if (temp == null)
        return head_3012;
    if (position_3012 == 1) {
        head_3012 = temp.next_3012;
        return head_3012;
    }

    for (int i_3012 = 1; temp != null && i_3012 <
position_3012; i_3012++) {
        prev = temp;
        temp = temp.next_3012;
    }
    if (temp != null) {
        prev.next_3012 = temp.next_3012;
    } else {
        System.out.println("data tidak ada");
    }
    return head_3012;
}

public static void printList_3012(NodeSLL_2511533012
head_3012) {
    NodeSLL_2511533012 curr = head_3012;
    while (curr.next_3012 != null) {
        System.out.print(curr.data_3012 + "--->");
        curr = curr.next_3012;
    }
    if (curr.next_3012 == null) {
        System.out.print(curr.data_3012);
    }
    System.out.println();
}

public static void main(String[] args) {
    // TODO Auto-generated method stub
    NodeSLL_2511533012 head = new NodeSLL_2511533012(1);
    head.next_3012 = new NodeSLL_2511533012(2);
    head.next_3012.next_3012 = new NodeSLL_2511533012(3);
    head.next_3012.next_3012.next_3012 = new
NodeSLL_2511533012(4);
    head.next_3012.next_3012.next_3012.next_3012 = new
NodeSLL_2511533012(5);
    head.next_3012.next_3012.next_3012.next_3012.next_3012
= new NodeSLL_2511533012(6);

    System.out.println("list awal : ");
    printList_3012(head);
}

```

```

        head = deleteHead_3012(head);
        System.out.println("List setelah head di hapus : ");
        printList_3012(head);

        head = removeLastNode_3012(head);
        System.out.println("List setelah simpul terakhir di
hapus : ");
        printList_3012(head);

        int position_3012 = 2;
        head = deleteNode_3012(head, position_3012);

        System.out.println("List setelah posisi 2 dihapus : ");
        printList_3012(head);
    }
}

```

Output :

```

<terminated> HapusSL_2511533012 [Java Application] C:\P
list awal :
1--->2--->3--->4--->5--->6
List setelah head di hapus :
2--->3--->4--->5--->6
List setelah simpul terakhir di hapus :
2--->3--->4--->5
List setelah posisi 2 dihapus :
2--->4--->5

```

Analisis :

Program ini digunakan untuk melakukan operasi **penghapusan node** pada Single Linked List.

Operasi yang tersedia meliputi:

- deleteHead untuk menghapus node pertama
- removeLastNode untuk menghapus node terakhir
- deleteNode untuk menghapus node pada posisi tertentu

Setelah proses penghapusan, program menampilkan isi linked list terbaru.

Kesimpulan: Program ini menjelaskan proses penghapusan node pada linked list.

BAB III

KESIMPULAN

3.1 Ringkasan

Pada praktikum pekan ke-5, mahasiswa mempelajari implementasi Single Linked List menggunakan Java. Praktikum mencakup pembuatan node, penambahan data, penghapusan data, traversal, dan pencarian data. Mahasiswa juga memahami hubungan antar node menggunakan pointer atau referensi.

3.2 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa Single Linked List merupakan struktur data dinamis yang memudahkan proses penambahan dan penghapusan data. Dengan memahami konsep node dan hubungan antar node, mahasiswa dapat mengelola data secara lebih fleksibel dibandingkan array. Praktikum ini menjadi dasar penting dalam mempelajari struktur data lanjutan.

DAFTAR PUSTAKA

- [1] Oracle, "The Java Tutorials," 2023. [Online]. Available: <https://docs.oracle.com/javase/tutorial/>.
- [2] H. M. Deitel dan P. J. Deitel, Java: How to Program, 10th ed. Boston: Pearson, 2015

Tugas Pratikum

1. Pasien_2511533012.java

```
package Pekan5_2511533012;

class Pasien_2511533012 {
    private String namaPasien_3012;
    private String penyakit_3012;
    private int nomorAntrian_3012;
    private Pasien_2511533012 next_3012;

    public Pasien_2511533012(String namaPasien_3012, String
penyakit_3012, int nomorAntrian_3012) {
        this.namaPasien_3012 = namaPasien_3012;
        this.penyakit_3012 = penyakit_3012;
        this.nomorAntrian_3012 = nomorAntrian_3012;
        this.next_3012 = null;
    }

    public String getNamaPasien_3012() {
        return namaPasien_3012;
    }

    public String getPenyakit_3012() {
        return penyakit_3012;
    }

    public int getNomorAntrian_3012() {
        return nomorAntrian_3012;
    }

    public Pasien_2511533012 getNext_3012() {
        return next_3012;
    }

    // Setter
    public void setNamaPasien_3012(String namaPasien_3012) {
        this.namaPasien_3012 = namaPasien_3012;
    }

    public void setPenyakit_3012(String penyakit_3012) {
        this.penyakit_3012 = penyakit_3012;
    }

    public void setNomorAntrian_3012(int nomorAntrian_3012) {
        this.nomorAntrian_3012 = nomorAntrian_3012;
    }

    public void setNext_3012(Pasien_2511533012 next_3012) {
        this.next_3012 = next_3012;
    }
}
```



```
}
```

2. RumahSakit_2511533012

```
package Pekan5_2511533012;
import java.util.Scanner;

public class RumahSakit_2511533012 {
    static Pasien_2511533012 head_3012 = null;
    static int counter_3012 = 0;

    public static void daftarkanPasien_3012(String nama_3012, String
keluhan_3012) {
        counter_3012++;

        Pasien_2511533012 pasienBaru_3012 =
            new Pasien_2511533012(nama_3012, keluhan_3012,
counter_3012);

        if (head_3012 == null) {
            head_3012 = pasienBaru_3012;
        } else {
            Pasien_2511533012 temp_3012 = head_3012;

            while (temp_3012.getNext_3012() != null) {
                temp_3012 = temp_3012.getNext_3012();
            }

            temp_3012.setNext_3012(pasienBaru_3012);
        }

        System.out.println("Pasien berhasil didaftarkan! Nomor Antrian:
"
            + pasienBaru_3012.getNomorAntrian_3012());
    }

    public static void panggilPasien_3012() {
        if (head_3012 == null) {
            System.out.println("Antrian kosong!");
        } else {
            System.out.println("Pasien dipanggil:");
            System.out.println("Nomor Antrian : " +
head_3012.getNomorAntrian_3012());
            System.out.println("Nama Pasien : " +
head_3012.getNamaPasien_3012());
            System.out.println("Keluhan : " +
head_3012.getPenyakit_3012());

            head_3012 = head_3012.getNext_3012();
        }
    }
}
```

```

public static void tampilkanAntrian_3012() {
    if (head_3012 == null) {
        System.out.println("Antrian masih kosong!");
    } else {
        Pasien_2511533012 temp_3012 = head_3012;
        int posisi_3012 = 1;

        System.out.println("=== DAFTAR ANTRIAN PASIEN ===");

        while (temp_3012 != null) {
            System.out.println("Posisi Antrian : " + posisi_3012);
            System.out.println("Nomor Antrian : " +
temp_3012.getNomorAntrian_3012());
            System.out.println("Nama Pasien : " +
temp_3012.getNamaPasien_3012());
            System.out.println("Keluhan : " +
temp_3012.getPenyakit_3012());
            System.out.println("-----");

            temp_3012 = temp_3012.getNext_3012();
            posisi_3012++;
        }
    }
}

public static void cariPasien_3012(String namaCari_3012) {
    if (head_3012 == null) {
        System.out.println("Antrian kosong!");
    } else {
        Pasien_2511533012 temp_3012 = head_3012;
        boolean ditemukan_3012 = false;

        while (temp_3012 != null) {
            if (temp_3012.getNamaPasien_3012()
                .equalsIgnoreCase(namaCari_3012)) {

                System.out.println("Pasien ditemukan!");
                System.out.println("Nomor Antrian : "
                    + temp_3012.getNomorAntrian_3012());
                System.out.println("Nama Pasien : "
                    + temp_3012.getNamaPasien_3012());
                System.out.println("Keluhan : "
                    + temp_3012.getPenyakit_3012());

                ditemukan_3012 = true;
                break;
            }

            temp_3012 = temp_3012.getNext_3012();
        }

        if (!ditemukan_3012) {

```

```

        System.out.println("Pasien tidak ditemukan!");
    }
}

public static void cekStatusAntrian_3012() {
    if (head_3012 == null) {
        System.out.println("Antrian kosong!");
    } else {
        Pasien_2511533012 temp_3012 = head_3012;
        int jumlah_3012 = 0;

        while (temp_3012 != null) {
            jumlah_3012++;
            temp_3012 = temp_3012.getNext_3012();
        }

        System.out.println("Jumlah Pasien Dalam Antrian : " +
jumlah_3012);
        System.out.println("Pasien Terdepan : "
+ head_3012.getNamaPasien_3012());
    }
}

public static void main(String[] args) {
    Scanner input_3012 = new Scanner(System.in);
    int pilihan_3012;

    do {
        System.out.println("\n=== Antrian Rumah Sakit NIM:
2511533012 ===");
        System.out.println("1. Daftarkan Pasien (Insert)");
        System.out.println("2. Panggil Pasien (Delete Head)");
        System.out.println("3. Tampilkan Antrian (Display)");
        System.out.println("4. Cari Pasien (Search)");
        System.out.println("5. Cek Status Antrian");
        System.out.println("6. Keluar");
        System.out.print("Pilihan : ");

        pilihan_3012 = input_3012.nextInt();
        input_3012.nextLine();

        switch (pilihan_3012) {
            case 1:
                System.out.print("Masukkan Nama Pasien : ");
                String nama_3012 = input_3012.nextLine();

                System.out.print("Masukkan Keluhan : ");
                String keluhan_3012 = input_3012.nextLine();

                daftarkanPasien_3012(nama_3012, keluhan_3012);
                break;

```

```

        case 2:
            panggilPasien_3012();
            break;

        case 3:
            tampilkanAntrian_3012();
            break;

        case 4:
            System.out.print("Masukkan Nama Pasien yang
Dicari : ");
            String cari_3012 = input_3012.nextLine();

            cariPasien_3012(cari_3012);
            break;

        case 5:
            cekStatusAntrian_3012();
            break;

        case 6:
            System.out.println("Program selesai.");
            break;

        default:
            System.out.println("Pilihan tidak valid!");
    }

    } while (pilihan_3012 != 6);
}

```

3. Outpun Syntax program

```

=== Antrian Rumah Sakit NIM: 2511533012 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan : 1
Masukkan Nama Pasien : Pikhul
Masukkan Keluhan : atit ati
Pasien berhasil didaftarkan! Nomor Antrian: 1

=== Antrian Rumah Sakit NIM: 2511533012 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan : 1
Masukkan Nama Pasien : annafi
Masukkan Keluhan : sipilis
Pasien berhasil didaftarkan! Nomor Antrian: 2

```

```
=== Antrian Rumah Sakit NIM: 2511533012 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan : 3
=== DAFTAR ANTRIAN PASIEN ===
Posisi Antrian : 1
Nomor Antrian  : 1
Nama Pasien   : Pikhul
Keluhan       : atit ati
-----
Posisi Antrian : 2
Nomor Antrian  : 2
Nama Pasien   : annafi
Keluhan       : sipilis
-----
```

```
=== Antrian Rumah Sakit NIM: 2511533012 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan : 4
Masukkan Nama Pasien yang Dicari : annafi
Pasien ditemukan!
Nomor Antrian : 2
Nama Pasien   : annafi
Keluhan       : sipilis
```

```
=== Antrian Rumah Sakit NIM: 2511533012 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan : 5
Jumlah Pasien Dalam Antrian : 2
Pasien Terdepan : Pikhul
```

```
=== Antrian Rumah Sakit NIM: 2511533012 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan : 2
Pasien dipanggil:
Nomor Antrian : 1
Nama Pasien   : Pikhul
Keluhan       : atit ati
```

```
=== Antrian Rumah Sakit NIM: 2511533012 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan : 2
Pasien dipanggil:
Nomor Antrian : 2
Nama Pasien   : annafi
Keluhan       : sipilis
```

Penjelasan:

Program ini dibuat untuk mensimulasikan sistem antrian pasien di rumah sakit menggunakan konsep Single Linked List. Dalam program ini, setiap pasien disimpan ke dalam sebuah node yang berisi data nama pasien, keluhan penyakit, nomor antrian, dan pointer yang menunjuk ke pasien berikutnya. Konsep linked list digunakan agar data pasien dapat ditambahkan dan dihapus secara dinamis tanpa harus menentukan ukuran data sejak awal.

Pada program terdapat dua kelas utama, yaitu kelas Pasien_2511533012 dan kelas RumahSakit_2511533012. Kelas Pasien_2511533012 berfungsi sebagai ADT atau representasi node dalam linked list. Kelas ini memiliki atribut berupa nama pasien, penyakit, nomor antrian, dan pointer next_3012 yang digunakan untuk menghubungkan node satu dengan node berikutnya. Selain itu, kelas ini juga memiliki constructor, getter, dan setter untuk mengatur dan mengambil data pasien.

Kelas RumahSakit_2511533012 berfungsi sebagai driver program yang mengatur seluruh proses antrian pasien. Pada kelas ini terdapat variabel head_3012 yang digunakan sebagai penunjuk node pertama dalam linked list serta variabel counter_3012 untuk memberikan nomor antrian secara otomatis kepada setiap pasien baru.

Method daftarkanPasien_3012() digunakan untuk menambahkan pasien baru ke dalam antrian. Proses penambahan dilakukan di bagian akhir linked list atau disebut Insert at Tail. Jika linked list masih kosong, maka node baru akan langsung menjadi head. Namun jika sudah terdapat data, program akan menelusuri node hingga bagian terakhir kemudian menyambungkan node baru ke node terakhir tersebut. Dengan cara ini, urutan pasien tetap sesuai dengan sistem FIFO (First In First Out).

Method panggilPasien_3012() digunakan untuk memanggil pasien yang berada di posisi paling depan. Program akan menampilkan data pasien terdepan kemudian menghapus node tersebut dengan cara memindahkan head ke node

berikutnya. Proses ini disebut Delete Head karena node pertama dihapus dari linked list.

Method `tampilkanAntrian_3012()` digunakan untuk menampilkan seluruh data pasien yang berada di dalam antrian. Program akan menelusuri linked list mulai dari head hingga node terakhir menggunakan perulangan `while`. Setiap data pasien seperti nomor antrian, nama pasien, dan keluhan akan ditampilkan secara berurutan.

Method `cariPasien_3012()` digunakan untuk mencari data pasien berdasarkan nama. Program akan melakukan pencarian dari node pertama hingga node terakhir menggunakan metode `equalsIgnoreCase()` sehingga pencarian tidak membedakan huruf besar dan kecil. Jika data ditemukan maka informasi pasien akan ditampilkan, sedangkan jika tidak ditemukan program akan menampilkan pesan bahwa pasien tidak ada di dalam antrian.

Method `cekStatusAntrian_3012()` digunakan untuk mengetahui kondisi antrian saat ini. Program akan menghitung jumlah seluruh pasien dalam linked list dan menampilkan informasi pasien yang berada di posisi paling depan. Jika linked list kosong maka program akan menampilkan pesan bahwa antrian kosong.

Pada method `main`, program menggunakan menu interaktif agar pengguna dapat memilih operasi yang ingin dijalankan. Pengguna dapat menambahkan pasien, memanggil pasien, menampilkan seluruh antrian, mencari pasien, mengecek status antrian, dan keluar dari program. Dengan adanya menu ini, program menjadi lebih mudah digunakan dan dapat mensimulasikan sistem antrian rumah sakit secara sederhana.